

SAP UI5 & Fiori Beginners Guide

Digital Personas Pvt Ltd

- Introduction To SAPUI5 1 - 5
- Landscape of SAPUI5
- Clear Understanding on End to End Data flow
- Initial flow of Execution of SAPUI5
- Introduction to Views and Controllers
- Difference between SAPUI5 and Fiori
- Routing in SAPUI5
- Basic UI Elements in SAPUI5
- Introduction to different Layouts
- Introduction to UI Controls

- Validations Types
- Message Handling and Types
- Different Types of Models
- Different Types of Bindings
- Different Types Of Views
- Life Cycle Methods/Hook Methods
- Passing Data from one view to another
- Playing with complex UI Elements with Json Data
- Working with radio buttons , Check Box , Value Helps
- Working with table and list UI Elements

- Introduction to Popup , Dialog , Fragments
- Working With ODATA Model
- Introduction to CRUD Application
- Types of Bindings using ODATA Model
- Filtering Data using ODATA Model
- Real Time Example with ODATA Model
- Introduction to File Upload Functionality
- SAPUI5 END – END Project
- Introduction to ODATA and Gateway system
- SAP Gate way system and its architecture

- Introduction to SAP Fiori
- Overview on SAP Fiori design guidelines
- Introduction to custom Fiori application and standard Fiori application.
- Introduction to Activation of Fiori Apps
- Introduction of Fiori Apps Library
- Introduction of S4 HANA
- CDS View Creation in HANA DB
- Overriding Backend Annotations in SAP UI5 App
- Clear Understanding of UI Annotations
- Working with Facets
- Deep dive in Fiori Elements Applications

Introduction to SAPUI5

➤ What is SAPUI5

SAPUI5 (SAP User Interface for HTML5) is a JavaScript-based framework developed by SAP for building enterprise-grade web applications. It is used to create responsive and interactive web applications that can run on desktop, tablet, and mobile devices. SAPUI5 is an open-source framework and is built on top of the popular JavaScript library, jQuery.

SAPUI5 provides a collection of pre-built UI elements, such as buttons, tables, forms, and charts, which can be easily integrated into web applications. It also offers a range of development tools and libraries, including the SAPUI5 development toolkit, which allows developers to create custom UI elements and themes.

SAPUI5 applications are typically built using the Model-View-Controller (MVC) design pattern, which helps to separate the application's data, presentation, and logic layers. This approach promotes modularization, maintainability, and reusability of code.

SAPUI5 is often used to build enterprise applications for SAP systems, such as SAP S/4HANA, but it can also be used to build standalone web applications.

➤ Previous technologies before SAPUI5

Before SAPUI5, SAP had several other technologies that were used to build web applications for their enterprise systems. Some of these technologies include:

1. **Web Dynpro:** Web Dynpro is a web application development technology provided by SAP. It is based on the Model-View-Controller (MVC) design pattern and allows developers to create UIs for web applications using drag and drop tools.

Introduction to SAPUI5

2. **BSP (Business Server Pages):** BSP is a server-side scripting technology used to build web applications for SAP systems. It uses a combination of HTML and ABAP code to generate dynamic web pages.

3. **SAPGUI (SAP Graphical User Interface):** SAPGUI is a desktop application that provides users with a graphical interface to access SAP systems. It is a thick client application that requires installation on the user's computer.

4. **Java-based technologies:** SAP also had several Java-based technologies for building web applications, including Java Server Pages (JSP), JavaServer Faces (JSF), and Java Web Dynpro.

However, with the increasing demand for modern and responsive web applications, SAPUI5 was introduced as a new and advanced technology for building web applications that can run on any device and platform.

➤ Disadvantages of previous technologies before sapui5

While the previous technologies used by SAP for web application development had their strengths, they also had some significant disadvantages, including:

1. **Limited cross-platform support:** Many of the previous SAP technologies were limited in their ability to run on multiple platforms and devices. For example, SAPGUI is a desktop application that requires installation on the user's computer, while BSP is a server-side scripting technology that generates HTML pages, which may not be optimized for mobile devices.
2. **Complex development process:** Many of the previous technologies required a high level of technical knowledge and experience to develop web applications. This often resulted in longer development times and higher costs.

3. **Limited design capabilities:** Many of the previous technologies did not provide robust design capabilities for web applications. This often resulted in web applications with a limited user interface and poor user experience.

4. **Limited scalability:** Some of the previous technologies were not designed to scale easily, which could result in performance issues for large-scale web applications.

SAPUI5 was developed to address many of these limitations and provide a modern and flexible framework for building enterprise-grade web applications that are responsive, scalable, and easy to develop and maintain.

➤ **Advantages of Sapui5 and how it overcomes the disadvantages of previous technologies**

SAPUI5 has several advantages over previous SAP technologies for web application development. Here are some of the key advantages of SAPUI5 and how it overcomes the disadvantages of previous technologies:

1. **Cross-platform support:** SAPUI5 is based on HTML5, CSS3, and JavaScript, which makes it platform-agnostic. It can run on any device with a modern web browser, including desktops, tablets, and mobile devices. This overcomes the limitations of previous technologies like SAPGUI and BSP, which were limited to specific platforms or devices.
2. **Responsive design:** SAPUI5 provides a range of pre-built UI elements and themes that are optimized for a modern and responsive design. This allows developers to create web applications with a consistent look and feel across different devices and platforms. This overcomes the limited design capabilities of previous technologies like BSP and Java-based technologies.

Introduction to SAPUI5

- 3. **Modular and reusable code:** SAPUI5 is based on the Model-View-Controller (MVC) design pattern, which promotes modularization and reusability of code. This allows developers to create scalable and maintainable web applications with less effort. This overcomes the complexity and limited scalability of previous technologies like Java-based technologies.
- 4. **Open source:** SAPUI5 is an open-source framework, which means that developers can access the source code and modify it to suit their specific needs. This allows for greater flexibility and customization compared to previous technologies like SAPGUI and BSP.
- 5. **Integration with SAP systems:** SAPUI5 is tightly integrated with SAP systems, including SAP S/4HANA, which makes it easy to develop web applications that integrate with SAP systems. This allows developers to create web applications that can access and manipulate data from SAP systems.

Overall, SAPUI5 provides a modern and flexible framework for building enterprise-grade web applications that are responsive, scalable, and easy to develop and maintain. It overcomes many of the limitations of previous SAP technologies for web application development and provides a powerful tool for developers to create web applications that meet the needs of modern businesses.

➤ Architecture of SAP UI5

The architecture of SAPUI5 is based on the Model-View-Controller (MVC) design pattern, which promotes separation of concerns between the data model, user interface, and business logic. The MVC pattern consists of three main components:

- 1. **Model:** The model represents the data and the business logic of the application. It includes data structures, data access layers, and business logic components.
- 2. **View:** The view represents the user interface of the application. It includes UI elements like buttons, input fields, and tables that are used to display the data to the user.

Introduction to SAPUI5

3. **Controller:** The controller acts as an intermediary between the model and the view. It handles user events and updates the model and view accordingly.

In SAPUI5, the architecture is divided into several layers, each of which has a specific role in the application. These layers are:

1. **Core layer:** The core layer includes the SAPUI5 library and provides the foundation for building SAPUI5 applications. It includes the core framework, runtime environment, and utility functions.

2. **UI layer:** The UI layer includes the UI controls, which are the building blocks for creating the user interface of the application. The UI controls include a wide range of predefined controls like input fields, tables, and charts, as well as custom controls that can be created by developers.

3. **Model layer:** The model layer includes the data model and the data binding mechanism. The data model represents the data that the application works with, and the data binding mechanism binds the data to the user interface elements.

4. **Service layer:** The service layer includes the data access and communication mechanisms. It is responsible for fetching and updating data from external sources like SAP systems or RESTful web services.

5. **Application layer:** The application layer includes the actual application code, which is developed by the developer. This layer includes the controllers and views that implement the business logic and user interface of the application.

Overall, the architecture of SAPUI5 is designed to promote modularity, maintainability, and scalability of the application code. It provides a flexible and extensible framework for building enterprise-grade web applications that can run on any device and integrate with SAP systems.

System Landscape of SAPUI5

The landscape of SAPUI5 Fiori applications typically consists of multiple layers, each with a specific role in the development, deployment, and operation of the applications. The layers can be divided into three main categories:

1. **Development layer:** The development layer includes the tools and infrastructure used by developers to build and test Fiori applications. This layer typically includes a development environment like SAP Web IDE or Eclipse, which provides an integrated development environment (IDE) for building Fiori applications. The development layer also includes version control systems like Git, which are used to manage the source code of the applications.
2. **Quality assurance layer:** The quality assurance layer includes the infrastructure used to test and validate Fiori applications before they are deployed to production. This layer includes testing tools like Selenium or QUnit, which are used to test the functionality of the applications. It also includes a quality assurance system, which is used to validate the applications against a set of quality criteria.
3. **Production layer:** The production layer includes the infrastructure used to deploy and operate Fiori applications in a live environment. This layer includes the application server, which hosts the Fiori applications, as well as load balancers and other infrastructure components that are used to ensure high availability and scalability of the applications.

In addition to these layers, the landscape of SAPUI5 Fiori applications also includes several other components, including:

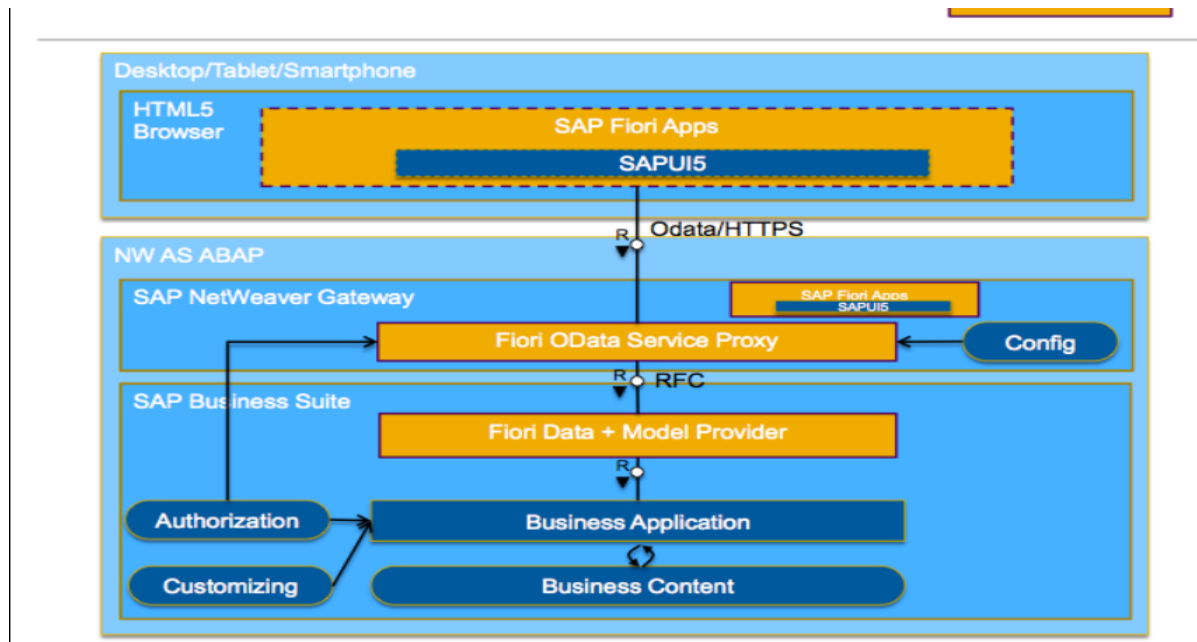
1. **SAP Gateway:** SAP Gateway is used to connect Fiori applications to backend systems like SAP ERP or SAP S/4HANA. It provides a unified API layer that abstracts the complexity of the backend systems and provides a simple and consistent interface for Fiori applications to access the data.

System Landscape of SAPUI5

2. **Fiori Launchpad:** Fiori Launchpad is a portal that provides a unified entry point for all Fiori applications. It provides a consistent user experience across all applications and allows users to launch and manage applications from a single location.

3. **SAP Cloud Platform:** SAP Cloud Platform provides a cloud-based infrastructure for deploying and operating Fiori applications. It provides a scalable and flexible environment that can be used to deploy applications in a public or private cloud.

Overall, the landscape of SAPUI5 Fiori applications is complex and includes many different components and layers. However, by leveraging the tools and infrastructure provided by SAP, developers can build and deploy enterprise-grade Fiori applications that provide a modern and intuitive user experience.



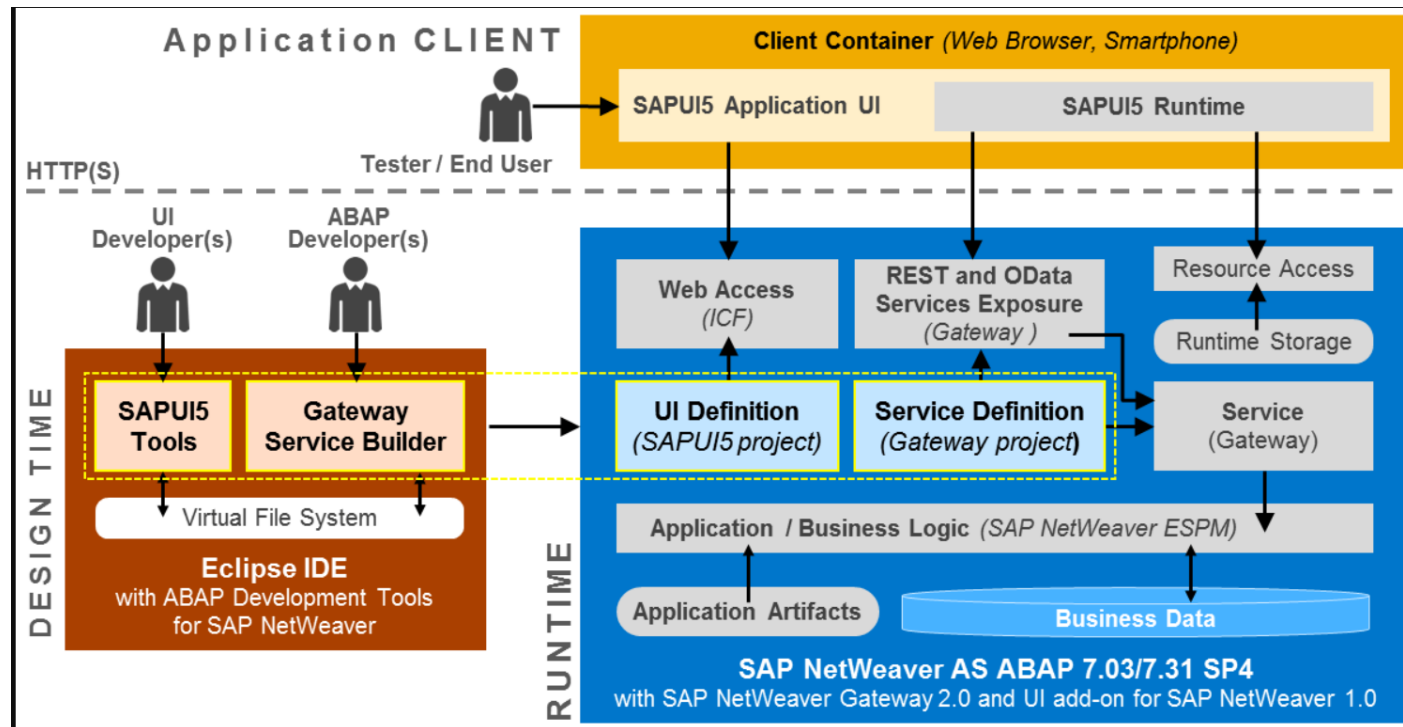
End to End Data Flow of SAPUI5

The end-to-end data flow in SAPUI5 apps refers to the process by which data is transferred from backend systems like SAP ERP or SAP S/4HANA to the user interface of the application. The data flow can be divided into several steps:

1. **Data Retrieval:** The first step in the data flow is to retrieve the data from the backend system. This can be done using SAP Gateway, which provides a unified API layer for accessing data from backend systems. Developers can define data models in SAPUI5 that map to the backend data structures, and use the data binding mechanism to bind the data to the user interface elements.
2. **Data Processing:** Once the data is retrieved from the backend system, it may need to be processed or transformed before it can be displayed in the user interface. This can be done using JavaScript functions or custom data processors that are defined in the application code.
3. **Data Binding:** The next step is to bind the processed data to the user interface elements. This is done using the data binding mechanism, which allows developers to define bindings between the data model and the user interface elements. The data binding mechanism ensures that the user interface is automatically updated whenever the data in the model changes.
4. **User Interaction:** Once the data is bound to the user interface elements, the user can interact with the application by entering data or triggering actions. The user interaction may result in changes to the data model, which are automatically propagated to the user interface elements.
5. **Data Submission:** When the user submits the data, it is sent back to the backend system for processing. This can be done using SAP Gateway, which provides a simple and consistent API for submitting data to backend systems. The data may need to be validated or transformed before it can be submitted, which can be done using JavaScript functions or custom data processors.
6. **Error Handling:** Finally, the application must handle errors that may occur during the data flow process. Errors can be handled using error handling functions or custom error processors that are defined in the application code. The application may also display error messages to the user to provide feedback on the error.

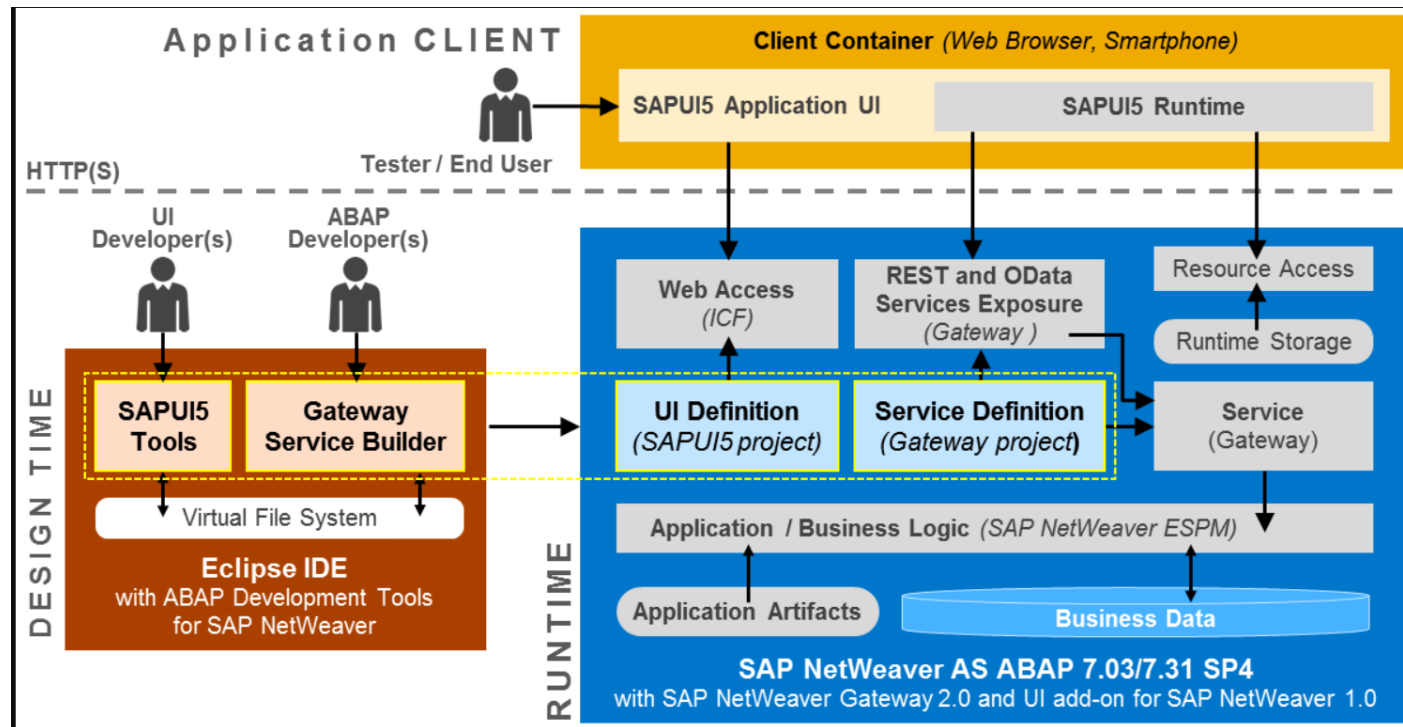
End to End Data Flow in SAPUI5

Overall, the end-to-end data flow in SAPUI5 apps is a complex process that involves multiple steps and components. By using the data binding mechanism and SAP Gateway, developers can build applications that provide a modern and intuitive user experience, while leveraging the power and flexibility of SAP backend systems.



End to End Data Flow in SAPUI5

Overall, the end-to-end data flow in SAPUI5 apps is a complex process that involves multiple steps and components. By using the data binding mechanism and SAP Gateway, developers can build applications that provide a modern and intuitive user experience, while leveraging the power and flexibility of SAP backend systems.



Difference Between SAPUI5 and Fiori Apps

SAPUI5 is a development toolkit for building web applications using HTML5, JavaScript, and CSS. It provides a rich set of UI controls and features that can be used to build responsive and intuitive user interfaces. SAPUI5 can be used to build standalone web applications or integrated with SAP systems like SAP ERP or SAP S/4HANA using SAP Gateway.

Fiori applications, on the other hand, are a specific type of SAPUI5 application that is designed to provide a consistent and modern user experience across different SAP systems. Fiori applications follow a set of design principles and guidelines that ensure a consistent look and feel, as well as a unified user experience. Fiori applications typically leverage SAP Gateway to access data from backend systems and provide a responsive and intuitive user interface.

Some of the key differences between SAPUI5 and Fiori applications are:

1. **Design:** SAPUI5 provides a toolkit for building web applications, while Fiori applications follow a specific design language and set of guidelines that ensure a consistent user experience across different applications.
2. **Functionality:** SAPUI5 provides a rich set of UI controls and features that can be used to build different types of applications, while Fiori applications are focused on providing specific business functions and processes.
3. **Integration:** SAPUI5 can be used to build standalone web applications or integrated with SAP systems using SAP Gateway, while Fiori applications are designed to integrate with SAP systems and provide access to backend data.
4. **User Experience:** Fiori applications are specifically designed to provide a modern and intuitive user experience, while SAPUI5 applications may vary in terms of their user interface and user experience.

Overall, SAPUI5 and Fiori applications are related but distinct concepts. While SAPUI5 provides the toolkit for building web applications, Fiori applications are a specific type of SAPUI5 application that follows a set of design principles and guidelines to provide a consistent and modern user experience across different SAP systems.

Initial Flow of Execution in SAPUI5

The initial flow of execution in SAPUI5 is as follows:

1. When the application is launched, the **“Component.js”** file is executed. This file is responsible for initializing the application and registering the necessary resources with SAPUI5.
2. The **“init”** function in **“Component.js”** is called. This function is responsible for creating the application and setting up the router.
3. The **“manifest.json”** file is loaded. This file contains the configuration for the application, including the routes, views, and models.
4. The router is initialized based on the configuration in the **“Manifest.json”** file. This allows the application to handle navigation between different views.
5. The first view specified in the routing configuration is displayed. This is typically the home view of the application.
6. The controller for the view is instantiated and the **“oninit”** function is called. This function is typically used to perform any initialization that is required for the view, such as setting up data models.
7. If the view contains any data-bound controls, such as tables or lists, the data for these controls is loaded. This is typically done using a model that is defined in the **“manifest.json”** file.
8. The view is rendered and displayed on the screen.
9. The user can interact with the view and navigate to other views using the router.
10. When the user navigates to a new view, the process repeats from step 6 onwards.

Overall, the initial flow of execution in SAPUI5 is driven by the **“component.js”** file, the **“manifest.json”** file, and the router. These components work together to initialize the application, handle navigation, and display the views.

Introduction to Views and Controllers in SAPUI5

➤ Introduction to Views and Controllers

Views and controllers are fundamental building blocks of SAPUI5 applications. Views are responsible for defining the structure and layout of the user interface, while controllers are responsible for defining the behavior and logic of the application.

In SAPUI5, views are defined using XML or JavaScript code and are responsible for rendering the user interface. A view can contain one or more UI elements, such as buttons, input fields, lists, and charts, and can be nested to create complex and dynamic user interfaces. Views are also responsible for binding data to the UI elements, which allows the application to display and update data in real-time.

Controllers, on the other hand, are responsible for defining the behavior and logic of the application. A controller is associated with a view and contains methods that are called in response to user interactions or programmatic events. Controllers can manipulate the data and UI elements in the view, interact with backend systems using APIs, and perform other operations to control the behavior of the application.

Here are the key features of views and controllers in SAPUI5:

1. Views are defined using XML or JavaScript code and are responsible for rendering the user interface.
2. Controllers are associated with views and contain methods that define the behavior and logic of the application.
3. Views and controllers can communicate with each other using events and methods.
4. Views can be nested to create complex and dynamic user interfaces.
5. Controllers can interact with backend systems using APIs, such as OData or REST.
6. Views and controllers are independent of each other and can be developed and tested separately.

Introduction to Views and Controllers in SAPUI5

Here is an example of a simple SAPUI5 view and controller:

View (XML):

```
<mvc:View xmlns:mvc="sap.ui.core.mvc" controllerName="myController">
  <Input id="inputField" value="{/firstName}"/>
  <Button id="saveButton" text="Save" press="onSave"/>
</mvc:View>
```

Controller (JavaScript):

```
sap.ui.define([
  "sap/ui/core/mvc/Controller"
], function(Controller) {
  "use strict";

  return Controller.extend("myController", {
    onSave: function() {
      // get the input value
      var inputValue = this.getView().byId("inputField").getValue();

      // do something with the input value
      // ...
    }
  });
});
```

In this example, the view contains an input field and a button, and the controller defines a function named **“onSave”** that is called when the button is clicked. The function retrieves the value of the input field and performs some action with it.

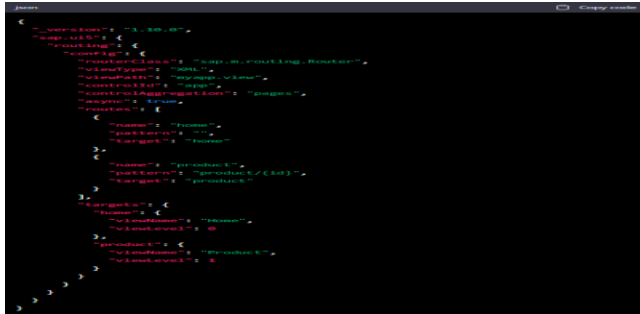
By understanding how views and controllers work together in SAPUI5, developers can create complex and dynamic user interfaces that meet the needs of their users.

➤ Routing

Routing is a mechanism used in SAPUI5 to manage navigation between different views of an application. It enables users to move from one view to another based on user actions or programmatic events.

Here's an example of how to use routing in SAPUI5:

First, we need to define the routing configuration in the “Manifest.Json” file



In this example, we define two routes: “home” and “product”. The “home” route is an empty pattern and the “product” route has a pattern of “product/{id}”. We also define two targets: “home” and “product”, which correspond to the views that will be displayed when the routes are navigated to.

Next, we need to create the views that correspond to the targets in the routing configuration. For example, we might have the following views:

Routing in SAPUI5

XML:

```
xml
Copy code

<mvc:View xmlns:mvc="sap.ui.core.mvc" xmlns="sap.m" controllerName="myapp.controller.Home">
  <Page title="Home">
    <Button text="View Product" press="onViewProductPress"/>
  </Page>
</mvc:View>

<mvc:View xmlns:mvc="sap.ui.core.mvc" xmlns="sap.m" controllerName="myapp.controller.Product">
  <Page title="Product">
    <Text text="Product {id}"/>
  </Page>
</mvc:View>
```

In this example, we define two views: “**Home**” and “**Product**”. The “**Home**” view contains a button that, when pressed, will navigate to the “**Product**” view with the appropriate product ID.

Finally, we need to implement the navigation logic in the controller for the “**Home**” view

In this example, we use the “**sap.ui.core.UIComponent.getRouterFor**” method to get the router, and then call the “**navTo**” method to navigate to the “**Product**” route with the product ID parameter set to “123”.

When the “**Product**” view is displayed, the controller for that view can access the product ID parameter using the “**getModel**” method

```
javascript
Copy code

sap.ui.define([
  "sap/ui/core/mvc/Controller"
], function(Controller) {
  "use strict";

  return Controller.extend("myapp.controller.Home", {
    onViewProductPress: function() {
      var oRouter = sap.ui.core.UIComponent.getRouterFor(this);
      oRouter.navTo("product", { id: 123 });
    }
  });
});
```

```
javascript
Copy code

sap.ui.define([
  "sap/ui/core/mvc/Controller",
  "sap/ui/core/routing/History"
], function(Controller, History) {
  "use strict";

  return Controller.extend("myapp.controller.Product", {
    onViewProductPress: function() {
      var oRouter = sap.ui.core.UIComponent.getRouterFor(this);
      oRouter.navTo("product", { id: 123 });
    }
  });
});
```

➤ Event Handling

Event handling is an important concept in SAPUI5 that allows developers to define and handle user interactions with UI elements in a web application. In SAPUI5, events can be triggered by user actions, such as clicking a button or selecting an item from a list, or by programmatic actions, such as updating data in the backend system.

Here are the steps to handle events in SAPUI5:

1. Define the UI element: First, define the UI element that will trigger the event.

For example, a button with an “id” of “saveButton” : **<Button id=“saveButton” text=“save”/>**

2. Define the event handler: Next, define the function that will handle the event when it is triggered. For example, a function named “onSave” that will be called when the "saveButton" is clicked:

“onSave: function(){// handle the event here}”

3. Attach the event handler: Finally, attach the event handler to the UI element using the “attachEvent” method. For example, to attach the “onSave” function to the "press" event of the "saveButton": “

this.getView().byId(“saveButton”).attachPress(this.Onsave);

Here is an example that demonstrates how to handle the "press" event of a button in SAPUI5:

Example:

```
<Button id="saveButton" text="Save" press="onSave"/>
```

```
onSave: function() {  
    // get the input value  
    var inputValue = this.getView().byId("inputField").getValue();  
  
    // do something with the input value  
    // ...  
}
```

In this example, the “**onSave**” function is called when the user clicks the "saveButton". The function retrieves the value of an input field with an “**id**” of "inputField" and performs some action with it. SAPUI5 provides a wide range of event types and APIs that can be used to handle different types of events in a web application. By understanding how to handle events in SAPUI5, developers can create interactive and responsive user interfaces that meet the needs of their users.

SAPUI5 provides a wide range of UI elements that can be used to build responsive and intuitive user interfaces. Some of the key UI elements in SAPUI5 include:

1. **Buttons:** Buttons are used to trigger an action or navigate to a different page. SAPUI5 provides different types of buttons, including standard buttons, icon buttons, and action buttons.
2. **Texts:** Texts are used to display static or dynamic text in the user interface. SAPUI5 provides several types of text elements, including labels, titles, and formatted texts.
3. **Input Fields:** Input fields are used to accept user input in the form of text, numbers, dates, and other types of data. SAPUI5 provides different types of input fields, including text fields, password fields, date pickers, and combo boxes.
4. **Lists:** Lists are used to display data in a tabular or hierarchical format. SAPUI5 provides different types of lists, including standard lists, tree tables, and responsive tables.
5. **Dialogs:** Dialogs are used to display messages or gather user input in a popup window. SAPUI5 provides different types of dialogs, including message dialogs, confirmation dialogs, and value help dialogs.
6. **Charts:** Charts are used to display data in a graphical format, such as line charts, bar charts, pie charts, and scatter charts. SAPUI5 provides a wide range of charting libraries and APIs to build interactive and responsive charts.

Here are some examples of SAPUI5 UI elements:

1. **Button :** `<Button text = "save press = "onSave" />`
2. **Text :** `<Text text= "Welcome to Digital Personas" />`
3. **Input Field :** `<Input Type = "Text" Value = "First Name" Placeholder = "Enter First name"/>`
4. **Dialog :** `<Dialog title = " Add Employee" type= "Message" state="Success"/>`
5. **Chart:** `<VizFrame VizType="column" dataset="{/salesSet}"/>`

These are just a few examples of the many UI elements available in SAPUI5. Developers can combine these elements to create complex and dynamic user interfaces that meet the needs of their users.

Introduction to Different Layouts

➤ Different Layouts in SAPUI5

SAPUI5 provides a variety of layout controls that allow developers to create responsive and flexible user interfaces. Here are some of the commonly used layout controls in SAPUI5:

1. HorizontalLayout: This layout control arranges its child controls horizontally in a row. It is useful when you want to display a series of controls in a single row.

Example:

```
<mvc:View xmlns:mvc="sap.ui.core.mvc" controllerName="myController">
  <layout:HorizontalLayout xmlns:layout="sap.ui.layout">
    <Button text="Button 1"/>
    <Button text="Button 2"/>
    <Button text="Button 3"/>
  </layout:HorizontalLayout>
</mvc:View>
```

This example shows how to use the HorizontalLayout control to arrange three buttons horizontally in a row.

2. VerticalLayout: This layout control arranges its child controls vertically in a column. It is useful when you want to display a series of controls in a single column.

Example :

```
<mvc:View xmlns:mvc="sap.ui.core.mvc" controllerName="myController">
```

```
  <layout:VerticalLayout xmlns:layout="sap.ui.layout">
    <Label text="First Name"/>
    <Input value="{/firstName}"/>
    <Label text="Last Name"/>
    <Input value="{/lastName}"/>
    <Label text="Email"/>
    <Input value="{/email}"/>
  </layout:VerticalLayout>
```

```
</mvc:View>
```

This example shows how to use the VerticalLayout control to arrange three labels and input fields vertically in a column.

Introduction to Different Layouts

3. **Grid:** This layout control allows you to arrange controls in a grid-like structure. You can define the number of rows and columns, as well as the size of each cell.

Example:

```
<mvc:View xmlns:mvc="sap.ui.core.mvc" controllerName="myController">
  <layout:Grid xmlns:layout="sap.ui.layout">
    <layout:content>
      <Button text="Button 1"/>
      <Button text="Button 2"/>
      <Button text="Button 3"/>
      <Button text="Button 4"/>
      <Button text="Button 5"/>
      <Button text="Button 6"/>
    </layout:content>
    <layout:layoutData>
      <layout:GridData span="L2 M3 S6"/>
      <layout:GridData span="L2 M3 S6"/>
      <layout:GridData span="L2 M3 S6"/>
      <layout:GridData span="L2 M3 S6"/>
      <layout:GridData span="L2 M3 S6"/>
      <layout:GridData span="L2 M3 S6"/>
    </layout:layoutData>
  </layout:Grid>
</mvc:View>
```

This example shows how to use the Grid control to arrange six buttons in a grid-like structure. The “Span” property of the “GridData” control defines the size of each cell.

Introduction to Different Layouts

4. **FlexBox:** This layout control allows you to create flexible layouts that adjust to the size of the screen. You can define the direction of the layout (horizontal or vertical), as well as the alignment and distribution of the child controls.

Example:

```
<mvc:View xmlns:mvc="sap.ui.core.mvc" controllerName="myController">
  <layout:FlexBox xmlns:layout="sap.ui.layout" direction="Column">
    <Label text="First Name"/>
    <Input value="{/firstName}"/>
    <Label text="Last Name"/>
    <Input value="{/lastName}"/>
    <Label text="Email"/>
    <Input value="{/email}"/>
  </layout:FlexBox>
</mvc:View>
```

This example shows how to use the FlexBox control to arrange three labels and input fields vertically in a column. The **“direction”** property of the FlexBox control defines the direction of the layout.

5. **Form:** This layout control allows you to create a form-like structure, where each control is labeled and arranged in a grid-like structure. It is useful when you want to collect data from the user.

Example:

```
<mvc:View xmlns:mvc="sap.ui.core.mvc" controllerName="myController">
  <layout:Form xmlns:layout="sap.ui.layout" editable="true">
    <layout:formContainers>
      <layout:FormContainer>
        <layout:formElements>
          <layout:FormElement label="First Name">
            <Input value="{/firstName}"/>
          </layout:FormElement>
          <layout:FormElement label="Last Name">
            <Input value="{/lastName}"/>
          </layout:FormElement>
        </layout:formElements>
      </layout:FormContainer>
    </layout:formContainers>
  </layout:Form>
</mvc:View>
```

Introduction to Different Layouts

6. ResponsiveGridLayout: This layout control allows you to create responsive layouts that adjust to the size of the screen. It is similar to the Grid control, but it automatically adjusts the number of columns based on the screen size.

7. SimpleForm: This layout control allows you to create a simple form-like structure, where each control is labeled and arranged in a column. It is useful when you want to display a small amount of data.

Example:

```
<mvc:View xmlns:mvc="sap.ui.core.mvc" controllerName="myController">
  <layout:SimpleForm xmlns:layout="sap.ui.layout" editable="true" layout="ResponsiveGridLayout">
    <layout:content>
      <Label text="First Name" />
      <Input value="{/firstName}" />
      <Label text="Last Name" />
      <Input value="{/lastName}" />
      <Label text="Email" />
      <Input value="{/email}" />
    </layout:content>
    <layout:ResponsiveGridLayout columnsL="2" columnsM="2" columnsS="1" labelSpanL="2" labelSpanM="2" labelSpanS="12" emptySpanL="0"
emptySpanM="0" emptySpanS="0" />
  </layout:SimpleForm>
</mvc:View>
```

In this example, we create a SimpleForm layout control and set the “**editable**” property to “**true**” to make the form editable. We also set the “**layout**” property to “**ResponsiveGridLayout**” to use the responsive grid layout.

Next, we add the form elements (labels and input fields) to the “**content**” aggregation of the SimpleForm control. Each form element is added to the content aggregation as a separate control.

Finally, we set the layout options for the responsive grid layout using the “**ResponsiveGridLayout**” control that's defined in the “**Layout**” aggregation of the SimpleForm control. In this example, we set the number of columns for each screen size (‘**columnsL**’, ‘**columnsM**’, “**columnsS**”), the number of columns that a label should span. This is just one example of how to use the SimpleForm layout control in SAPUI5. SimpleForm also supports other types of layouts such as grid, responsive grid, and responsive layout.

Introduction to UI Controls

➤ UI Controls

SAPUI5 provides different controls to build user interfaces, including the Panel, IconTabBar, and ObjectHeader controls. Here's a brief introduction to each of these controls:

1. **Panel Control:** The Panel control is used to group and display content in a bordered container. It can be used to structure and organize a page, and can contain other controls such as forms, tables, or charts.

Example:

```
<Panel headerText="Employee Details" expandable="true" expanded="true">
  <content>
    <Text text="Name: John Doe"/>
    <Text text="Age: 30"/>
    <Text text="Address: 123 Main St"/>
  </content>
</Panel>
```

2. **IconTabBar Control:** The IconTabBar control is used to display multiple tabs, where each tab contains a different set of content. It can be used to display different views or to organize information into categories. Each tab can have its own icon, text, or both.

Example:

```
<IconTabBar>
  <items>
    <IconTabFilter text="Employees" icon="sap-icon://employee">
      <content>
        <List>
          <StandardListItem title="John Doe" description="30 years old"/>
          <StandardListItem title="Jane Smith" description="25 years old"/>
        </List>
      </content>
    </IconTabFilter>
    <IconTabFilter text="Projects" icon="sap-icon://activity-items">
      <content>
        <List>
          <StandardListItem title="Project A" description="Description of project A"/>
          <StandardListItem title="Project B" description="Description of project B"/>
        </List>
      </content>
    </IconTabFilter>
  </items>
</IconTabBar>
```

Introduction to UI Controls

3. ObjectHeader Control: The ObjectHeader control is used to display information about a single object, such as a customer, an employee, or a product. It can display the object's name, image, description, and other details.

Example :

```
<ObjectHeader title="John Doe" numberUnit="USD">
  <attributes>
    <ObjectAttribute title="Age" text="30"/>
    <ObjectAttribute title="Address" text="123 Main St"/>
  </attributes>
  <statuses>
    <ObjectStatus text="Active" state="Success"/>
  </statuses>
  <image src="https://cdn.pixabay.com/photo/2016/08/08/09/17/avatar-1577909_1280.png"/>
</ObjectHeader>
```

In the above example, the ObjectHeader control is used to display information about a person named John Doe. The "title" property is used to set the person's name, while the "attributes" property is used to display additional information such as age and address. The "Statuses" Property is used to display the person's status, while the "image" property is used to display an image of the person.

Validation Types

➤ Types of Validations

In SAPUI5, there are several types of validations that can be used to ensure that user input is correct and complete. Some of the most common validation types include:

1.Required Field Validation: This type of validation ensures that a user has entered a value in a required field. It can be implemented using the "mandatory" property in input fields.

Ex:

```
<Input value="{/name}" placeholder="Enter your name" mandatory="true" />
```

In this example, the "mandatory" property is set to "true" to ensure that the user enters a value in the name field.

2.Length Validation: This type of validation ensures that user input is of the correct length. It can be implemented using the "maxLength" property in input fields.

EX:

```
<Input value="{/phone}" placeholder="Enter your phone number" maxLength="10" />
```

In this example, the "maxLength" property is set to "10" to ensure that the user enters a phone number of exactly 10 digits.

3.Pattern Validation: This type of validation ensures that user input matches a specific pattern, such as a phone number or email address. It can be implemented using the "valueState" property in input fields.

Ex:

```
<Input value="{/email}" placeholder="Enter your email address" valueStateText="Please enter a valid email address">
  <layoutData>
    <FlexItemData growFactor="1" />
  </layoutData>
  <core:customData>
    <core:CustomData key="sap-ui-custom-settings" value='{"inputValidation": {"type": "email"}}' writeToDom="true" />
  </core:customData>
</Input>
```

Validation Types

In this example, the "valueState" property is used to set the validation state of the input field to "Error" if the entered email address does not match the email pattern.

4.Range Validation:This type of validation ensures that user input falls within a specific range of values. It can be implemented using the "min" and "max" properties in input fields.

EX:

```
<Input value="{/age}" placeholder="Enter your age" type="Number" min="18" max="100" />
```

In this example, the "min" and "max" properties are set to "18" and "100", respectively, to ensure that the user enters an age between 18 and 100.

5.Custom Validation:This type of validation allows you to create your own validation logic using JavaScript code. It can be implemented using the "validate" event in input fields.

```
<Input value="{/password}" placeholder="Enter your password" type="Password" liveChange="onLiveChange" />
onLiveChange: function(event) {
    var password = event.getParameter("value");
    var regex = /^(?=.*\d)(?=.*[a-z])(?=.*[A-Z])[0-9a-zA-Z]{8,}$/;
    if (!regex.test(password)) {
        event.getSource().setValueState("Error");
        event.getSource().setValueStateText("Password must contain at least one uppercase letter, one lowercase letter, one digit, and be at least 8 characters long.");
    } else {
        event.getSource().setValueState("None");
        event.getSource().setValueStateText("");
    }
}
```

In this example, the "onLiveChange" function contains custom validation logic to ensure that the entered password contains at least one uppercase letter, one lowercase letter, one digit, and is at least 8 characters long. If the entered password does not meet these requirements, the input field's validation state is set to "Error" and an error message is displayed. If the entered password

Message Handling and Types

➤ Message Handling

In SAPUI5, message handling is an important part of creating robust and user-friendly applications. Message handling allows you to inform users of important information, errors, and warnings that may occur during application execution. There are several types of messages in SAPUI5 that you can use to provide feedback to users.

Here are the message types and how they can be used:

1. **Information Messages:** These messages provide information to the user about the status of the application. They are usually used to confirm successful execution of a task or to provide additional information to the user. They can be displayed using the `sap.m.MessageToast` control or the `sap.m.MessageStrip` control.

EX : `sap.m.MessageToast.show("Task successfully completed");`

2. **Success Messages:** These messages indicate successful execution of a task. They are usually displayed to the user after a task has been successfully completed. They can be displayed using the `sap.m.MessageToast` control or the `sap.m.MessageStrip` control.

EX: `sap.m.MessageToast.show("Task successfully completed");`

3. **Warning Messages:** These messages indicate that an issue has occurred during application execution, but it is not critical. They are usually used to alert users of potential issues that may require attention. They can be displayed using the `sap.m.MessageToast` control or the `sap.m.MessageStrip` control.

Ex: `sap.m.MessageToast.show("Please check your input, some fields are missing");`

4. **Error Messages:** These messages indicate that a critical error has occurred during application execution. They are usually used to alert users of issues that require immediate attention. They can be displayed using the `sap.m.MessageBox` control or the `sap.m.MessageStrip` control.

EX: `sap.m.MessageBox.error("An unexpected error occurred. Please try again later.");`

Message Handling and Types

5. Confirmation Messages: These messages ask the user to confirm an action before it is executed. They are usually used when a critical action is being taken, such as deleting data. They can be displayed using the `sap.m.MessageBox` control.

EX:

```
sap.m.MessageBox.confirm("Are you sure you want to delete this data?", {  
    title: "Confirm Deletion",  
    onClose: function(oAction) {  
        if (oAction === sap.m.MessageBox.Action.OK) {  
            // Delete data  
        }  
    }  
});
```

In SAPUI5, message handling can be implemented using various controls and APIs provided by the framework. These controls and APIs allow you to display messages in a consistent and user-friendly manner, and to handle user interaction with messages in a predictable way.

Types of Models

➤ Different Types of Models

SAPUI5 provides different types of models that can be used to represent data in an application. Here are some of the commonly used models in SAPUI5 along with examples:

1. **JSON Model:** The JSON model is used to represent data in JSON format. It is suitable for small to medium-sized data sets. Here is an example:

```
var oModel = new sap.ui.model.json.JSONModel({  
    "name": "John",  
    "age": 30,  
    "address": {  
        "city": "New York",  
        "state": "NY"  
    }  
});
```

2. **XML Model:** The XML model is used to represent data in XML format. It is suitable for medium to large-sized data sets. Here is an example:

```
var oModel = new sap.ui.model.xml.XMLModel("data.xml");
```

3. **OData Model:** The OData model is used to represent data from an OData service. It supports CRUD (create, read, update, delete) operations and data binding. Here is an example:

```
var oModel = new sap.ui.model.odata.v2.ODataModel("/my/odata/service", true);
```

4. **Resource Model:** The resource model is used to represent localized data such as labels and texts. It is suitable for multi-language applications. Here is an example:

```
var oModel = new sap.ui.model.resource.ResourceModel({  
    bundleUrl: "i18n/messageBundle.properties"  
});
```

In SAPUI5, models are used to store and manipulate data in an application. By using the appropriate model for your data, you can ensure that your application is efficient, responsive, and scalable.

Types of Binding

➤ Types of Binding

In SAPUI5, there are four types of binding:

1. Property Binding
2. Element Binding
3. Aggregation Binding
4. Composite Binding

Here are examples of each type of binding:

- 1. Property Binding:** Property binding is used to bind a single property of a control or an element to a model. In this type of binding, the value of the control property is bound to a single property of a data model.

EX: // create a model with data

```
var oModel = new sap.ui.model.json.JSONModel({  
  name: "John",  
  age: 30  
});
```

// set the model to a control

```
var oInput = new sap.m.Input({  
  value: "{/name}"  
});  
oInput.setModel(oModel);
```

In this example, the value of the Input control is bound to the "name" property of the JSON model.

Types of Binding

2. Element Binding: Element binding is used to bind a whole control or an element to a model. In this type of binding, the whole control is bound to a single object in the data model.

EX: // create a model with data

```
var oModel = new sap.ui.model.json.JSONModel({  
    name: "John",  
    age: 30  
});
```

// set the model to a control

```
var oForm = new sap.ui.layout.form.SimpleForm({  
    editable: true,  
    layout: "ResponsiveGridLayout",  
    labelSpanXL: 3,  
    emptySpanXL: 9,  
    columnsXL: 1,  
    content: [  
        new sap.m.Label({ text: "Name" }),  
        new sap.m.Input({ value: "{/name}" }),  
        new sap.m.Label({ text: "Age" }),  
        new sap.m.Input({ value: "{/age}" })  
    ]  
});
```

```
oForm.setModel(oModel);
```

In this example, the SimpleForm control is bound to the whole JSON model.

Types of Binding

3. Aggregation Binding: Aggregation binding is used to bind a control's aggregation to an array or a list in the model. In this type of binding, the control's aggregation is bound to a list of objects in the data model.

EX:

```
// create a model with data
var oModel = new sap.ui.model.json.JSONModel({
  items: [
    { name: "Item 1", price: 100 },
    { name: "Item 2", price: 200 },
    { name: "Item 3", price: 300 }
  ]
});
```

```
// set the model to a control
var oList = new sap.m.List({
  headerText: "Items",
  items: {
    path: "/items",
    template: new sap.m.StandardListItem({
      title: "{name}",
      description: "{price}"
    })
  }
});
oList.setModel(oModel);
```

In this example, the List control's "items" aggregation is bound to the "items" array in the JSON model.

Types of Binding

4. Composite Binding: Composite binding is used to bind multiple properties of a control to a model. In this type of binding, the properties of a control are bound to multiple properties of the data model.

Ex:

```
// create a model with data
var oModel = new sap.ui.model.json.JSONModel({
  firstName: "John",
  lastName: "Doe",
  address: {
    street: "123 Main St",
    city: "Anytown",
    state: "CA"
  }
});
```

```
// set the model to a control
var oPanel = new sap.m.Panel({
  headerText: "Person",
  content: [
    new sap.m.Label({ text: "First Name" }),
    new sap.m.Input({ value: "{/firstName}"

```

Types of Views

➤ Types of Views

SAPUI5 offers several types of views to structure the user interface. Here are some of the most commonly used views along with examples:

1. **XML View:** XML views use an XML file to describe the user interface. It's a declarative way of defining the UI structure.

EX: `<mvc:View xmlns:mvc="sap.ui.core.mvc" xmlns="sap.m"`

`controllerName="myController">`

`<Panel headerText="XML View Example">`

`<Label text="Name"/>`

`<Input value="{/name}"/>`

`<Label text="Age"/>`

`<Input value="{/age}"/>`

`</Panel>`

`</mvc:View>`

2. **JSON View:** JSON views use a JSON file to define the user interface. It's also a declarative way of defining the UI structure, but with a different syntax.

```

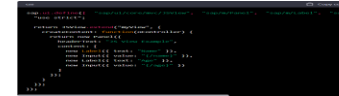
{
  "Type": "sap.ui.core.mvc.JSONView",
  "controllerName": "myController",
  "content": [
    {
      "Type": "sap.m.Panel",
      "headerText": "JSON View Example",
      "content": [
        {
          "Type": "sap.m.Label",
          "text": "Name",
          "value": "{/name}"
        },
        {
          "Type": "sap.m.Label",
          "text": "Age",
          "value": "{/age}"
        }
      ]
    }
  ]
}

```

In this example, a JSON view is used to define the same form as in the XML view example.

Types of Views

3. JavaScript View: JavaScript views use a JavaScript file to define the user interface. It allows more flexibility and programmability than the XML and JSON views.



In this example, a JavaScript view is used to define the same form as in the previous examples.

4. HTML View: HTML views use an HTML file to define the user interface. It's useful when the UI needs to be integrated with other HTML content.

```
<!DOCTYPE html>
<html>
<head>
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta charset="utf-8">
  <title>HTML View Example</title>
  <script id="sap-ui-bootstrap"
    type="text/javascript"
    src="https://sapui5.hana.ondemand.com/resources/sap-ui-core.js"
    data-sap-ui-libs="sap.m"
    data-sap-ui-theme="sap_bluecrystal"
    data-sap-ui-bindingsyntax="complex"></script>
  <script type="text/javascript">
    sap.ui.require(["sap/ui/core/componentContainer"], function(ComponentContainer) {
      new ComponentContainer({
        name: "myComponent",
        settings: {
          id: "myComponent"
        }
      }).placeAt("content");
    });
  </script>
</head>
</html>
```

LifeCycle Methods / Hook Methods

➤ Lifecycle Methods

In SAPUI5, hook methods are predefined methods that are provided by the framework and are intended to be implemented by developers to customize the behavior of various aspects of the framework. These hook methods are called by the framework at specific points during the lifecycle of the application or UI component. Here are some commonly used hook methods in SAPUI5:

1. **onInit:** This method is called when the component is initialized. Developers can use this method to perform any initialization that is required for the component.

```
EX: onInit: function() {  
    // perform initialization tasks  
},
```

In this example, the onInit method is used to perform any initialization that is required for the component.

2. **onBeforeRendering:** This method is called just before the component is rendered. Developers can use this method to perform any updates or modifications to the UI before it is displayed.

```
EX: onBeforeRendering: function() {  
    // perform updates or modifications to the UI  
}
```

In this example, the onBeforeRendering method is used to perform any updates or modifications to the UI before it is displayed.

3. **onAfterRendering:** This method is called just after the component is rendered. Developers can use this method to perform any post-rendering tasks, such as adding event handlers to the UI elements.

```
EX: onAfterRendering: function() {  
    // perform post-rendering tasks  
}
```

In this example, the onAfterRendering method is used to perform any post-rendering tasks, such as adding event handlers to the UI elements.

LifeCycle Methods / Hook Methods

4. **onExit**: This method is called when the component is destroyed. Developers can use this method to clean up any resources that were used by the component.

```
EX:onExit: function() {  
    // clean up any resources used by the component  
}
```

In this example, the onExit method is used to clean up any resources that were used by the component.

5. **onBeforeShow**: This method is called just before the component is displayed. Developers can use this method to perform any updates or modifications to the UI before it is shown.

```
EX: onBeforeShow: function() {  
    // perform updates or modifications to the UI  
}
```

In this example, the onBeforeShow method is used to perform any updates or modifications to the UI before it is shown.

6. **onAfterShow**: This method is called just after the component is displayed. Developers can use this method to perform any post-show tasks, such as adding event handlers to the UI elements.

```
EX:onAfterShow: function() {  
    // perform post-show tasks  
}
```

In this example, the onAfterShow method is used to perform any post-show tasks, such as adding event handlers to the UI elements.

By using these hook methods, developers can easily customize the behavior of their SAPUI5 applications and components to meet their specific needs.

Types of Passing Data from One view to Another

- In SAPUI5, there are several ways to pass data from one view to another. Here are some of the common methods with examples:

1. Passing data using Routing Parameters:

Routing parameters are used to pass data between views during navigation. You can define parameters in the route definition and set their values when navigating to the view. Here is an example:

Ex: **// define a route with a parameter in manifest.json**

```
"myRoute": {  
  "pattern": "myRoute/{parameter}",  
  "name": "myRoute",  
  "target": "myTarget"  
}
```

// navigate to the route with a parameter

```
var oRouter = sap.ui.core.UIComponent.getRouterFor(this);  
oRouter.navTo("myRoute", {  
  parameter: "Hello world!"  
});
```

In this example, we define a route "myRoute" with a parameter "{parameter}" and a target "myTarget". Then, when navigating to the route, we use the navTo method of the router to set the parameter value to "Hello world!".

Types of Passing Data from One view to Another

2. Passing data using Models:

Models are used to share data between views in SAPUI5. You can create a model in the root component and then bind it to the views. Here is an example:

```
EX: // create a model in the root component
var oModel = new sap.ui.model.json.JSONModel({
    message: "Hello world!"
});
this.setModel(oModel);
```

```
// bind the model to the view
var oView = this.getView();
oView.setModel(oModel);
```

In this example, we create a JSON model in the root component with a property "message" set to "Hello world!". Then, we set the model to the component using the setModel method and bind it to the view using the setModel method of the view.

3. Passing data using Event Bus:

The Event Bus is a mechanism for decoupling components in SAPUI5. You can publish an event in one component and subscribe to it in another component to pass data. Here is an example:

```
EX: // publish an event with data
var oEventBus = sap.ui.getCore().getEventBus();
oEventBus.publish("myEvent", {
    message: "Hello world!"
});

// subscribe to the event and receive the data
oEventBus.subscribe("myEvent", function(channel, event, data) {
    var message = data.message;
}, this);
```

In this example, we get the Event Bus instance and publish an event "myEvent" with data "message: Hello world!". Then, we subscribe to the event in another component and receive the data in the handler function.

These are some of the common methods of passing data between views in SAPUI5. You can choose the method based on the scenario and requirements.

Playing with Complex Ui Elements with Json Data

➤ Complex UI Elements with JSON Data

In SAPUI5, complex UI elements can be defined and configured using JSON data. These UI elements include tables, forms, and charts. Here is an introduction to these elements with examples of how to create them using JSON data:

1.Tables: Tables are commonly used to display tabular data in SAPUI5. You can define a table using JSON data and configure its columns, binding, and events. Here is an example:

Ex:

```
<mvc:View xmlns:mvc="sap.ui.core.mvc" xmlns="sap.ui.table" xmlns:m="sap.m"
  controllerName="myController">
  <Table id="myTable" visibleRowCount="10" selectionMode="Single"
    selectionBehavior="RowOnly" enableCellFilter="true" enableColumnReordering="true">
    <columns>
      <Column width="100px">
        <Label text="Product ID" />
        <template>
          <m:Text text="{ProductID}" />
        </template>
      </Column>
      <Column width="150px">
        <Label text="Product Name" />
        <template>
          <m:Text text="{ProductName}" />
        </template>
      </Column>
    </columns>
  </Table>
</mvc:View>
```

In this example, we define a table with two columns "Product ID" and "Product Name" and bind it to a data model. The table displays the data with selection mode set to single.

Playing with Complex Ui Elements with Json Data

2.List:

To create a list in a view, define the list in the XML view using the "sap.m.List" control and configure its items, data binding, and events. Here is an example:

```
<mvc:View xmlns:mvc="sap.ui.core.mvc" xmlns="sap.m"
  controllerName="myController">
  <List id="myList" mode="SingleSelect" headerText="Product List"
    items="{/products}" selectionChange="onSelectionChange">
    <items>
      <StandardListItem title="{ProductID}" description="{ProductName}" />
    </items>
  </List>
</mvc:View>
```

In this example, we define a list with items "Product ID" and "Product Name" and bind it to a data model. The list displays the data with selection mode set to single. The "selectionChange" event handler is defined in the controller and is executed when the user selects an item in the list.

3.Charts:

Charts are used to display graphical data in SAPUI5. You can define a chart using JSON data and configure its type, data series, and properties. Here is an example:

Playing with Complex Ui Elements with Json Data

```
// define a chart with data
var oChart = new sap.viz.ui5.controls.VizFrame({
  vizType: "column",
  dataset: new sap.viz.ui5.data.FlattenedDataset({
    dimensions: [{
      name: "Month",
      value: "{month}"
    }],
    measures: [{
      name: "Revenue",
      value: "{revenue}"
    }],
    data: {
      path: "/"
    }
  })
});
```

```
// bind the chart to data
var oModel = new sap.ui.model.json.JSONModel({
  data: [
    { month: "January", revenue: 1000 },
    { month: "February", revenue: 2000 },
    { month: "March", revenue: 1500 }
  ]
});
oChart.setModel(oModel);
oChart.bindData("/data");
```


Playing with Complex Ui Elements with Json Data

In this example, we define a column chart with one data series "Revenue" and bind it to a JSON model with data on three months. The "{month}" and "{revenue}"

➤ Introduction to radio buttons , Check Box , Value Helps examples

In SAPUI5, radio buttons, check boxes, and value helps are UI controls that allow users to make selections and input data. Here is an introduction to these controls with examples of how to create them:

1. Radio buttons:

Radio buttons are used when users need to select one option from a list of mutually exclusive options. You can define a group of radio buttons using a `sap.m.RadioButtonGroup` control and configure its options and events.

Here is an example:

```
// define a radio button group
var oRadioGroup = new sap.m.RadioButtonGroup({
    columns: 2,
    select: function(oEvent) {
        var selected = oEvent.getParameter("selectedItem").getText();
        console.log("Selected: " + selected);
    }
});

// add radio buttons to the group
oRadioGroup.addButton(new sap.m.RadioButton({
    text: "Option 1",
    selected: true
}));
oRadioGroup.addButton(new sap.m.RadioButton({
    text: "Option 2"
}));
oRadioGroup.addButton(new sap.m.RadioButton({
    text: "Option 3"
}));
```

➤ Types of Tables

In SAPUI5, there are different types of tables that can be used to display data in a tabular format. Some of the common types of tables are:

1. **Responsive Table:** A responsive table automatically adapts its layout to the available screen size. It is used for displaying a small amount of data and is ideal for mobile devices.

Ex:

```
<mvc:View controllerName="myController" xmlns="sap.m"
xmlns:mvc="sap.ui.core.mvc">
<Table id="myTable" items="{path: '/Products'}" mode="SingleSelectMaster">
  <columns>
    <Column>
      <Label text="Product Name"/>
      <template>
        <Text text="{ProductName}"/>
      </template>
    </Column>
    <Column>
      <Label text="Price"/>
      <template>
        <Text text="{Price}"/>
      </template>
    </Column>
  </columns>
</Table>
</mvc:View>
```

Types of Tables in sapui5

2. **Grid Table:** A grid table is used for displaying large amounts of data. It is a highly performant table and has advanced features like scrolling, sorting, and filtering.

Ex:

```
<mvc:View controllerName="myController" xmlns="sap.ui.table"
xmlns:mvc="sap.ui.core.mvc">
<Table id="myTable" rows="{path: '/Products'}" visibleRowCount="10">
<columns>
<Column>
<label>
<Text text="Product Name"/>
</label>
<template>
<Text text="{ProductName}"/>
</template>
</Column>
<Column>
<label>
<Text text="Price"/>
</label>
<template>
<Text text="{Price}"/>
</template>
</Column>
</columns>
</Table>
</mvc:View>
```

3. **Tree Table:** A tree table is used for displaying hierarchical data in a tree-like structure. It is similar to a grid table but has an additional column for displaying the tree hierarchy.

Ex:

```
<mvc:View controllerName="myController" xmlns="sap.ui.table"
xmlns:mvc="sap.ui.core.mvc">
<TreeTable id="myTable" rows="{path: '/Products'}" visibleRowCount="10">
<columns>
<Column>
<label>
<Text text="Product Name"/>
</label>
<template>
<Text text="{ProductName}"/>
</template>
</Column>
<Column>
<label>
<Text text="Price"/>
</label>
<template>
<Text text="{Price}"/>
</template>
</Column>
</columns>
<extension>
<m:Toolbar>
<m:Button text="Expand All" press="onExpandAll"/>
<m:Button text="Collapse All" press="onCollapseAll"/>
</m:Toolbar>
</extension>
<HierarchicalColumn>
<Column>
<label>
<Text text="Category"/>
</label>
<template>
<Text text="{Category}"/>
</template>
</Column>
</HierarchicalColumn>
</TreeTable>
</mvc:View>
```

4. **Analytical Table:** An analytical table is used for displaying analytical data. It has advanced features like aggregation, grouping, and drilling down.

Ex:

```
<mvc:View controllerName="myController" xmlns="sap.ui.table"
xmlns:mvc="sap.ui.core.mvc">
<AnalyticalTable id="myTable" rows="{path: '/Products'}" visibleRowCount="10">
<columns>
```

➤ Types of Lists

In SAPUI5, there are several types of lists that can be used to display data in different formats. Some of the commonly used types of lists are:

1. **Standard List:** This is a simple list containing a set of items arranged vertically. Each item in the list can have one or more sub-items, and the list can be made scrollable if there are more items than can fit on the screen.

EX: `<List id="list" items="{/items}">`
 `<StandardItem title="{Name}" description="{Description}"/>`
`</List>`

2. **Responsive Table:** This is a table that adjusts its layout based on the device size and orientation. It can have columns that can be sorted, filtered and grouped, and it can also be made scrollable if there are more rows than can fit on the screen.

```
<Table id="table" items="{/items}">
  <columns>
    <Column>
      <Text text="Name"/>
    </Column>
    <Column>
      <Text text="Description"/>
    </Column>
  </columns>
  <items>
    <ColumnListItem>
      <Text text="{Name}"/>
      <Text text="{Description}"/>
    </ColumnListItem>
  </items>
</Table>
```

3.Tree: This is a hierarchical list where each item can have one or more child items. It is useful for displaying data in a nested format, such as a file directory.

```
<Tree id="tree" items="{/items}">
    <StandardTreelItem title="{Name}">
        <Tree items="{path:'childItems'}">
            <StandardTreelItem title="{Name}" />
        </Tree>
    </StandardTreelItem>
</Tree>
```

4. Grid List: This is a list where items are arranged in a grid format with a fixed number of items per row. It is useful for displaying items with images or icons.

```
<GridList id="gridList" items="{/items}" width="auto" >
    <tiles>
        <GridTile>
            <Image src="{ImageUrl}" width="100%" height="100%"/>
            <Text text="{Name}" />
        </GridTile>
    </tiles>
</GridList>
```

These are just a few examples of the types of lists that can be used in SAPUI5. There are many other types of lists and list-like controls available, each with their own unique features and use cases.

Introduction to radio buttons , Check Box , Value Helps examples

In this example, we define a radio button group with three options "Option 1", "Option 2", and "Option 3". The "select" event handler logs the selected option to the console.

2.Check boxes:

Check boxes are used when users need to select one or more options from a list of non-mutually exclusive options. You can define a group of check boxes using sap.m.CheckBox controls and configure their labels, values, and events. Here is an example:

```
// define a group of check boxes
var oCheckBox1 = new sap.m.CheckBox({
    text: "Option 1",
    selected: true,
    select: function() {
        console.log("Option 1 selected: " + this.getSelected());
    }
});
var oCheckBox2 = new sap.m.CheckBox({
    text: "Option 2",
    select: function() {
        console.log("Option 2 selected: " + this.getSelected());
    }
});

// add check boxes to a vertical layout
var oLayout = new sap.ui.layout.VerticalLayout({
    content: [oCheckBox1, oCheckBox2]
});
```


Introduction to radio buttons , Check Box , Value Helps examples

3.Value helps:

Value helps are used to provide users with a list of selectable values for a field. You can define a value help using a `sap.ui.comp.valuehelpdialog.ValueHelpDialog` control and configure its search help, selection mode, and events. Here is an example:

```
// define a value help dialog
var oValueHelpDialog = new sap.ui.comp.valuehelpdialog.ValueHelpDialog({
  title: "Select a Product",
  supportMultiselect: true,
  supportRanges: false,
  key: "ProductId",
  descriptionKey: "ProductName",
  ok: function(oEvent) {
    var aTokens = oEvent.getParameter("tokens");
    console.log("Selected tokens: " + aTokens);
  }
});
```

```
// add search help to the dialog
oValueHelpDialog.setRangeKeyFields([
  {
    label: "Product ID",
    key: "ProductId",
    type: "string"
  }, {
    label: "Product Name",
    key: "ProductName",
    type: "string"
  }
]);
```

```
// open the value help dialog
oValueHelpDialog.open();
```

In this example, we define a value help dialog with a list of products and a search help for filtering them. The "ok" event handler logs the selected tokens to the console when the dialog is closed.

Introduction to Popup , Dialog , Fragments

➤ Introduction to Popup

in SAPUI5, popup dialogs are used to display additional information or allow the user to perform some action on the current screen. Fragments are reusable UI elements that can be embedded in other views, controllers, or fragments. Here are examples of how to create popup dialogs and fragments in a view in SAPUI5:

1. Popup Dialog: To create a popup dialog in a view, define the dialog in a separate XML view using the "sap.m.Dialog" control and configure its content, events, and properties. Here is an example:

EX: <!-- The main view with a button to open the dialog -->

```
<mvc:View xmlns:mvc="sap.ui.core.mvc" xmlns="sap.m" controllerName="myController">
  <Button text="Open Dialog" press="onOpenDialog" />
</mvc:View>
```

<!-- The dialog view with some content and a button to close the dialog -->

```
<mvc:View xmlns:mvc="sap.ui.core.mvc" xmlns="sap.m">
  <Dialog title="Popup Dialog" contentWidth="400px" contentHeight="300px" horizontalScrolling="false" verticalScrolling="false">
    <VBox>
      <Label text="Enter your name:" />
      <Input id="nameInput" />
    </VBox>
    <Button text="OK" press="onCloseDialog" />
  </Dialog>
</mvc:View>
```

In this example, we define a popup dialog in a separate XML view and use the "onOpenDialog" event handler to open the dialog. The dialog view contains a label and an input field to capture the user's name, as well as a button to close the dialog. The "onCloseDialog" event handler is defined in the controller and is executed when the user clicks the OK button to close the dialog.

Introduction to Popup , Dialog , Fragments

2.Fragment:

To create a fragment in a view, define the fragment in a separate XML view using the "sap.ui.core.Fragment" control and configure its content, events, and properties. Here is an example:

```
<!-- The main view with a button to open the fragment -->
<mvc:View xmlns:mvc="sap.ui.core.mvc" xmlns="sap.m" controllerName="myController">
  <Button text="Open Fragment" press="onOpenFragment" />
</mvc:View>

<!-- The fragment view with some content and a button to close the fragment -->
<mvc:View xmlns:mvc="sap.ui.core.mvc" xmlns="sap.m">
  <core:FragmentDefinition xmlns:core="sap.ui.core">
    <VBox>
      <Label text="Enter your name:" />
      <Input id="nameInput" />
    </VBox>
    <Button text="OK" press="onCloseFragment" />
  </core:FragmentDefinition>
</mvc:View>
```

In this example, we define a fragment in a separate XML view and use the "onOpenFragment" event handler to open the fragment. The fragment view contains a label and an input field to capture the user's name, as well as a button to close the fragment. The "onCloseFragment" event handler is defined in the controller and is executed when the user clicks the OK button to close the fragment. Note that we use the "sap.ui.core.Fragment" control to include the fragment in the main view.

Introduction to Fragments for reusable scenarios in sapui5

➤ Fragments for reusable scenarios

Fragments in SAPUI5 are reusable UI components that can be defined separately from views and can be included in different views or even other fragments. They can be used to create popups, dialogs, panels, and other reusable UI components.

Fragments have the same structure as views and consist of XML views, controllers, and JavaScript files. The difference is that fragments are not defined as standalone objects and do not have a separate namespace. Instead, they are created and used dynamically.

Using fragments in SAPUI5 has the following advantages:

- 1.Reusability: Fragments can be used across different views or other fragments, making it easy to create reusable UI components.
- 2.Separation of concerns: Fragments allow you to separate the UI component from the rest of the application logic, making it easier to maintain and modify the UI component.
- 3.Improved performance: Fragments can be loaded asynchronously, which can improve the performance of the application.

Here is an example of how to create a simple fragment in SAPUI5:

- 1.Create a new file called "myFragment.xml" and define the UI component inside it. For example, you can create a simple dialog box with some text and a button:

```
<core:FragmentDefinition xmlns="sap.m" xmlns:core="sap.ui.core">
  <Dialog title="My Dialog">
    <content>
      <Text text="Hello, world!"/>
    </content>
    <beginButton>
      <Button text="OK"/>
    </beginButton>
  </Dialog>
</core:FragmentDefinition>
```

Introduction to Fragments for reusable scenarios in sapui5

2. Create a new file called "myController.js" and define the controller for the fragment. For example, you can define a method to handle the "OK" button click event:

```
sap.ui.define([
    "sap/ui/core/mvc/Controller"
], function(Controller) {
    "use strict";

    return Controller.extend("myController", {
        onOK: function() {
            // Handle the "OK" button click event
        }
    });
});
```

To use the fragment in a view, you can include it in the XML view using the 'Fragment' Tag and specify the name of the fragment file and the controller for the fragment. For example:

```
<mvc:View xmlns:mvc="sap.ui.core.mvc" xmlns="sap.m">
    <Fragment name="myFragment" type="XML" />
</mvc:View>
```

To open the fragment, you can call the "**sap.ui.core.Fragment.load()**" method in the controller and specify the ID of the view and the name of the fragment file. For example:

```
sap.ui.define([
    "sap/ui/core/mvc/Controller",
    "sap/ui/core/Fragment"
], function(Controller, Fragment) {
    "use strict";

    return Controller.extend("myController", {
        onOpenDialog: function() {
            var oView = this.getView();

            Fragment.load({
                id: oView.getId(),
                name: "myFragment"
            }).then(function(oDialog) {
                oView.addDependent(oDialog);
                oDialog.open();
            });
        }
    });
});
```

Introduction to Fragments for reusable scenarios in sapui5

In this example, we define a method to open the fragment called **"onOpenDialog"**. This method loads the fragment using the **"sap.ui.fragment.load()"** method and specifies the ID of the view and the name of the fragment file. Once the fragment is loaded, we add it as a dependent of the view using the **'oView.addDependent()'** method and then open it using the **'oDialog.open()'** method.

Overall, fragments are a powerful feature of SAPUI5 that can help you create reusable and maintainable UI components in your application.

➤ Implementing F4 help functionality using Fragments for inputs in sapui5

implementing F4 help functionality using fragments for inputs in SAPUI5:

In the view file (XML view), define an input field with an attached value help:

```
<mvc:View xmlns:core="sap.ui.core" xmlns:mvc="sap.ui.core.mvc" xmlns="sap.m"
  controllerName="myController">
  <Input id="myInput" showValueHelp="true" valueHelpRequest="openValueHelp" />
  <core:Fragment fragmentName="myValueHelpDialog" type="XML" />
</mvc:View>
```

In the controller file, define the **'openValueHelp'** function to create and open the value help dialog:

Introduction to Fragments for reusable scenarios in sapui5

```
sap.ui.define([
    "sap/ui/core/mvc/Controller",
    "sap/ui/core/Fragment",
    "sap/ui/model/json/JSONModel"
], function(Controller, Fragment, JSONModel) {
    "use strict";

    return Controller.extend("myController", {
        openValueHelp: function() {
            var oView = this.getView();

            // create value help dialog lazily
            if (!this._oValueHelpDialog) {
                this._oValueHelpDialog = sap.ui.xmlfragment(oView.getId(), "myValueHelpDialog", this);
                oView.addDependent(this._oValueHelpDialog);
            }

            // open value help dialog
            this._oValueHelpDialog.open();
        },

        onValueHelpSearch: function(oEvent) {
            var sValue = oEvent.getParameter("value");
            var oFilter = new sap.ui.model.Filter("Name", sap.ui.model.FilterOperator.Contains, sValue);
            var oBinding = oEvent.getSource().getBinding("items");
            oBinding.filter([oFilter]);
        },

        onValueHelpClose: function(oEvent) {
            var oSelectedItem = oEvent.getParameter("selectedItem");
            if (oSelectedItem) {
                var sName = oSelectedItem.getTitle();
                var oInput = this.getView().byId("myInput");
                oInput.setValue(sName);
            }
            oEvent.getSource().getBinding("items").filter([]);
        }
    });
});
```

Introduction to Fragments for reusable scenarios in sapui5

```
sap.ui.define([
    "sap/ui/core/mvc/Controller",
    "sap/ui/core/Fragment",
    "sap/ui/model/json/JSONModel"
], function(Controller, Fragment, JSONModel) {
    "use strict";

    return Controller.extend("myController", {
        openValueHelp: function() {
            var oView = this.getView();

            // create value help dialog lazily
            if (!this._oValueHelpDialog) {
                this._oValueHelpDialog = sap.ui.xmlfragment(oView.getId(), "myValueHelpDialog", this);
                oView.addDependent(this._oValueHelpDialog);
            }

            // open value help dialog
            this._oValueHelpDialog.open();
        },

        onValueHelpSearch: function(oEvent) {
            var sValue = oEvent.getParameter("value");
            var oFilter = new sap.ui.model.Filter("Name", sap.ui.model.FilterOperator.Contains, sValue);
            var oBinding = oEvent.getSource().getBinding("items");
            oBinding.filter([oFilter]);
        },

        onValueHelpClose: function(oEvent) {
            var oSelectedItem = oEvent.getParameter("selectedItem");
            if (oSelectedItem) {
                var sName = oSelectedItem.getTitle();
                var oInput = this.getView().byId("myInput");
                oInput.setValue(sName);
            }
            oEvent.getSource().getBinding("items").filter([]);
        }
    });
});
```


Introduction to Fragments for reusable scenarios in sapui5

In the **'openValueHelp'** function, we first get the current view instance and then create the value help dialog lazily, i.e., only when it is needed for the first time. We use the **'sap.ui.xmlfragment'** method to create the dialog instance and pass the fragment name, type, and controller as parameters. Then we add the dialog as a dependent of the view so that the dialog is automatically destroyed when the view is destroyed.

In the **'onValueHelpSearch'** function, which is called when the user types in the search field of the value help dialog, we create a filter based on the entered value and apply it to the items binding of the dialog.

In the **'onValueHelpClose'** function, which is called when the user selects an item in the value help dialog, we get the selected item, extract its name, and set it as the value of the input field. We also clear the filter on the items binding.

In the fragment file, define the value help dialog content and bind the search and close buttons to the corresponding functions in the controller:

```
<core:FragmentDefinition xmlns="sap.m" xmlns:core="sap.ui.core">
  <Dialog title="Value Help" contentWidth="500px" class="sapUiContentPadding">
    <SearchField search="onValueHelpSearch" />
    <List items="{/Products}">
      <items>
        <StandardListItem title="{Name}" type="Active" press="onValueHelpClose" />
      </items>
    </List>
  </Dialog>
</Fragment>
```

Implementing F4 help functionality using Fragments for inputs

F4 help functionality is a common feature in SAPUI5 applications that enables users to search for values from a set of predefined values or a large data set. It is typically used with input fields to help users select valid values quickly and easily. In SAPUI5, F4 help functionality can be implemented using fragments.

Here's an example of how to implement F4 help functionality using fragments for input fields:

1. Create a fragment with a list or table control that displays the possible values for the input field.
2. In the controller of the view that contains the input field, define a function that opens the fragment as a dialog box when the user clicks on the input field. This function should create a new instance of the fragment, set the necessary model data, and bind the table or list control to the model.
3. In the fragment controller, define a function that handles the user's selection of a value from the list or table control. This function should close the dialog box and set the selected value as the value of the input field.

Here's the code example:

Fragment:

```
<core:FragmentDefinition xmlns="sap.m" xmlns:core="sap.ui.core">
  <List id="list" items="{/Values}">
    <items>
      <StandardListItem title="{Value}" press="onListItemPress" />
    </items>
  </List>
</core:FragmentDefinition>
```

Implementing F4 help functionality using Fragments for inputs

Controller:

```
onValueHelpRequest: function() {
    if (!this._valueHelpDialog) {
        this._valueHelpDialog = sap.ui.xmlfragment(
            "my.app.fragments.ValueHelpDialog",
            this
        );
        this.getView().addDependent(this._valueHelpDialog);
    }
    this._valueHelpDialog.open();
},

onValueHelpDialogClose: function(oEvent) {
    var oSelectedItem = oEvent.getParameter("selectedItem");
    if (oSelectedItem) {
        var sSelectedValue = oSelectedItem.getTitle();
        this.getView().byId("inputField").setValue(sSelectedValue);
    }
    oEvent.getSource().getBinding("items").filter([]);
},
```

Implementing F4 help functionality using Fragments for inputs

3 Fragment Controller

```
onListItemPress: function(oEvent) {  
    var oSelectedItem = oEvent.getSource();  
    var oDialog = oSelectedItem.getParent().getParent();  
    var oInput = sap.ui.getCore().byId(  
        oDialog.data("inputId")  
    );  
    oInput.setValue(oSelectedItem.getTitle());  
    oDialog.close();  
},
```

In the above example, a list control is used in the fragment to display the possible values for the input field. When the user clicks on the input field, the “**onValueHelpRequest**” function is called, which creates and opens the fragment dialog box. When the user selects a value from the list, the ‘**onListItemPress**’ function is called, which sets the selected value as the value of the input field and closes the dialog box.

➤ **complex Exercise on PopupDialogs in sapui5**

Create a master-detail application that displays a list of products in a table. When a product is selected from the table, a popup dialog should appear that displays the product details. The user should be able to edit the product details in the popup dialog and save the changes back to the model.

Steps to follow:

1. Create a new SAPUI5 application and add a JSON model that contains a list of products with properties such as name, description, price, and image URL.
2. Create a master-detail view with a table that displays the list of products from the model. When a product is selected from the table, store the selected product's index in the model.
3. Create a fragment that contains a form with input fields for each product property. When the fragment is opened, populate the input fields with the selected product's properties.
4. Add a "Edit" button to the detail view that opens the fragment when clicked. Pass the selected product's index to the fragment as a custom data property.
5. When the user clicks the "Save" button in the fragment, retrieve the updated values from the input fields and update the product object in the model.
6. Refresh the detail view with the updated product details.
7. Add a "Cancel" button to the fragment that closes the popup dialog without making any changes to the model.
8. Test the application to ensure that the popup dialog displays the correct product details, allows the user to edit and save changes, and cancels without making any changes to the model.

➤ Practical complex Exercise on Table and list UI elements

Create a SAPUI5 application that displays a list of products in a table. The table should have the following columns:

- Product ID
- Product Name
- Description
- Category
- Price

The application should have the following features:

- 1.The data for the table should be loaded from a JSON model. The JSON model should contain at least 10 products.
- 2.The user should be able to sort the table by clicking on the column headers.
- 3.The user should be able to filter the table by entering text in a search field.
- 4.The user should be able to select a row in the table and view more details about the product in a dialog box.
- 5.The user should be able to add a new product to the table using a form in a dialog box.
- 6.The user should be able to edit an existing product in the table using a form in a dialog box.
- 7.The user should be able to delete a product from the table.

Create a new view and add a `sap.ui.table.Table` control

➤ Practical complex Exercise on Table and list UI elements

Create a SAPUI5 application that displays a list of products in a table. The table should have the following columns:

- Product ID
- Product Name
- Description
- Category
- Price

The application should have the following features:

- 1.The data for the table should be loaded from a JSON model. The JSON model should contain at least 10 products.
- 2.The user should be able to sort the table by clicking on the column headers.
- 3.The user should be able to filter the table by entering text in a search field.
- 4.The user should be able to select a row in the table and view more details about the product in a dialog box.
- 5.The user should be able to add a new product to the table using a form in a dialog box.
- 6.The user should be able to edit an existing product in the table using a form in a dialog box.
- 7.The user should be able to delete a product from the table.

Create a new view and add a `sap.ui.table.Table` control